Artificial Neural Networks for Insurance Data

Sahar Chiha

April 12, 2025

Contents

1 Introduction **2**

2 Algorithms **3**
2.1 Feedforward Neural Network Overview 3
2.2 Combined Actuarial Neural Networks (CANNs) 5

3 Steps to Build the Model **7**
3.1 Feedforward Neural Network 7
3.2 Combined Actuarial Neural Network (CANNs) 9

4 Parameters and Hyperparameters **11**
4.1 Feedforward Neural Networks 11
4.2 Combined Actuarial Neural Networks (CANNs) 13

1 Introduction

Artificial Neural Networks (ANNs) are computational models inspired by the structure and function of biological neural networks. They are used for a variety of tasks, including pattern recognition, classification, regression, and prediction. ANNs consist of interconnected nodes (neurons) organized in layers: an input layer, one or more hidden layers, and an output layer. Each connection between neurons has a weight associated with it, which is adjusted during the learning process. For a probabilistic approach to neural networks in insurance, see Olsgärde (2021).

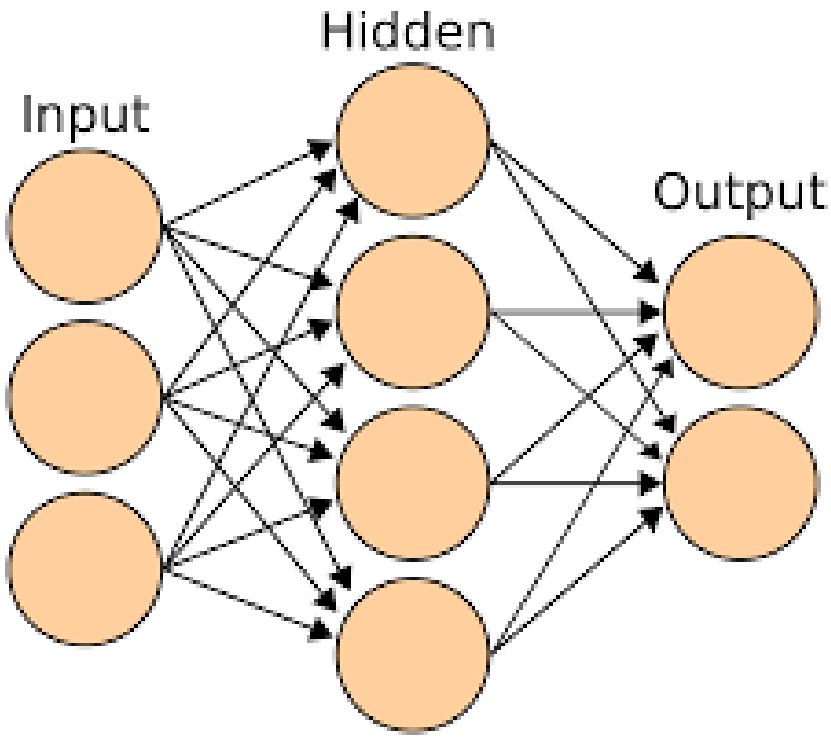

Figure 1: A simple diagram of an Artificial Neural Network

The power of ANNs lies in their ability to learn complex, nonlinear relationships from data, making them highly effective for tasks where traditional statistical methods may fall short. They are particularly useful in handling large datasets and complex feature interactions.

2 Algorithms

2.1 Feedforward Neural Network Overview

The feedforward neural network operates in two main phases: **feedforward propagation** and **backpropagation**. Below is the step-by-step algorithm:

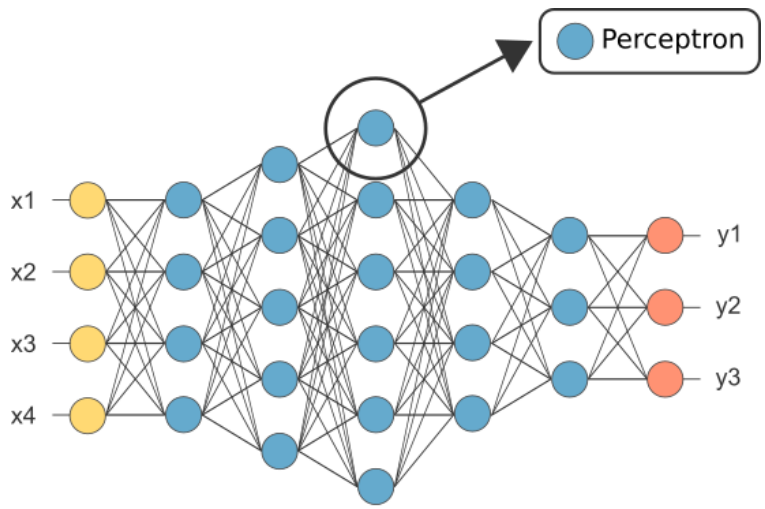

Figure 2: Feedforward Neural Network Architecture

1. Initialization

- **Input Data:** Prepare the input data $x = [x_1, x_2, \ldots, x_n]$.
- **Weights and Biases:** Initialize weights W and biases b randomly or using specific initialization methods (e.g., Xavier or He initialization).
- **Activation Function:** Choose activation functions for each layer (e.g., ReLU, sigmoid, tanh).

2. Feedforward Propagation

This phase computes the outputs of the network layer by layer.

- **Step 1: Input Layer** — Pass the input vector x to the first layer.
- **Step 2: Hidden Layers** — For each hidden layer:
  - Compute the weighted sum:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

where $z^{(l)}$ is the pre-activation value, $W^{(l)}$ is the weight matrix, $a^{(l-1)}$ is the activation from the previous layer, and $b^{(l)}$ is the bias vector.

- Apply the activation function:

$$a^{(l)} = f(z^{(l)})$$

- **Step 3: Output Layer** — Compute final output using the same process:

$$y = f(W^{(L)}a^{(L-1)} + b^{(L)})$$

3. Backpropagation

This phase adjusts weights and biases to minimize error using gradient descent.

- **Step 1: Compute Loss**

$$\text{Loss} = \mathcal{L}(y, t)$$

  Common loss functions include Mean Squared Error (MSE) or Cross-Entropy Loss.

- **Step 2: Gradient Calculation** For each layer, compute gradients of the loss with respect to weights and biases:

  - For weights:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = a^{(l-1)}\delta^{(l)}$$

  - For biases:

$$\frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}$$

  - Error term δ:

$$\delta^{(L)} = (y - t)f'(z^{(L)})$$
$$\delta^{(l)} = (W^{(l+1)}\delta^{(l+1)})f'(z^{(l)})$$

- **Step 3: Update Weights and Biases**

$$W^{(l)} = W^{(l)} - \eta\frac{\partial \mathcal{L}}{\partial W^{(l)}} \quad \text{and} \quad b^{(l)} = b^{(l)} - \eta\frac{\partial \mathcal{L}}{\partial b^{(l)}}$$

  where η is the learning rate.

4. Training Loop

Repeat feedforward and backpropagation for multiple epochs until convergence or satisfactory performance is achieved.

5. Prediction

After training, use the feedforward phase to predict outputs for new inputs.

Summary

The feedforward neural network algorithm involves propagating inputs forward through layers to compute predictions, calculating errors, and adjusting weights and biases via backpropagation to optimize performance.

2.2 Combined Actuarial Neural Networks (CANNs)

The Combined Actuarial Neural Network (CANN) algorithm integrates a classical actuarial model, such as a Generalized Linear Model (GLM), with a neural network to leverage the strengths of both approaches. Probabilistic approaches, such as those discussed by Olsgärde (2021), can further enhance CANNs. Below is the step-by-step algorithm:

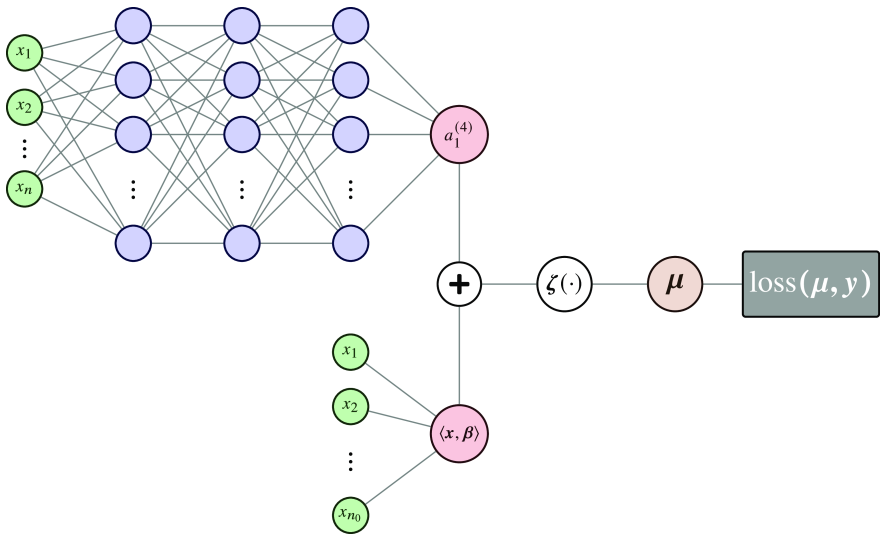

Figure 3: Architecture of a Combined Actuarial Neural Network (CANN)

1. **Initialization**
   - **Data Preparation:**
     - Input features include structured insurance data (e.g., policyholder characteristics, claim history) and additional telematics data if available.
     - Ensure proper encoding of categorical variables and scaling of numerical features.
   - **Model Components:**
     - Classical actuarial model: Typically a GLM (e.g., Poisson or Negative Binomial regression for claim frequency).
     - Neural network: Multilayer Perceptron (MLP) architecture.

2. Model Architecture

The CANN model consists of two components:

- **Classical Regression Component:**
  - Define the regression function $\lambda(x)$ using GLM:

$$\log(\lambda(x)) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots$$

  - This provides a solid foundation for interpretability.
- **Neural Network Component:**
  - Use an MLP to capture complex interactions and nonlinear relationships in the data.
  - The MLP processes telematics or high-dimensional inputs and outputs a correction term.
- **Blending:**
  - Combine the outputs of the GLM and MLP:

$$\log(\lambda(x)) = GLM(x) + NN(x)$$

  - This blending can be implemented via skip connections or additive layers.

3. Training Process

- **Step 1: Loss Function**

$$\mathcal{L} = -\sum_{i=1}^{n} (y_i \log(\lambda_i) - \lambda_i)$$

  This negative log-likelihood ensures compatibility with actuarial modeling principles.
- **Step 2: Gradient Descent**
  - Initialize parameters of the neural network using weights from the GLM.
  - Optimize both GLM and neural network components jointly using gradient descent.
- **Step 3: Regularization**
  - Apply regularization techniques like L2 penalty to avoid overfitting, especially for high-dimensional data.

4. Validation

- Split data into training and validation sets.
- Evaluate performance using insurance modeling metrics such as Mean Squared Error (MSE), deviance, or Akaike Information Criterion (AIC).

5. Prediction

For new observations, compute predictions using the combined model:

$$\lambda(x) = e^{GLM(x)+NN(x)}$$

6. Advantages of CANNs

- **Interpretability:** The GLM component provides actuarial insights into claim frequency drivers.
- **Flexibility:** The neural network captures complex nonlinear relationships and interactions.
- **Enhanced Predictive Accuracy:** By blending classical and modern approaches, CANNs outperform standalone models in many contexts.

This algorithm allows insurers to model claim frequency effectively while balancing interpretability and predictive power.

3 Steps to Build the Model

3.1 Feedforward Neural Network

Below are the detailed steps for constructing and training a feedforward neural network:

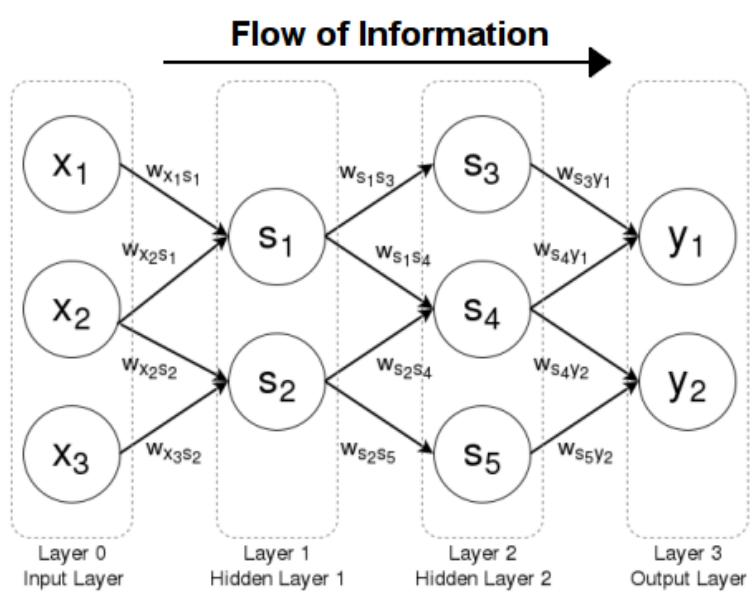

Figure 4: Steps to Build a Feedforward Neural Network

1. **Data Preparation**
   - **Collect Data:** Gather the dataset relevant to your task (e.g., insurance data for claim frequency).
   - **Preprocess Data:**
     - Normalize or scale numerical features to ensure consistent input ranges.
     - Encode categorical variables using techniques like one-hot encoding or embeddings.
     - Split the data into training, validation, and test sets.
2. **Define the Network Architecture**
   - **Input Layer:** Number of neurons = number of input features.
   - **Hidden Layers:** Add one or more hidden layers with nonlinear activation functions (ReLU, sigmoid, tanh).
   - **Output Layer:**
     - Regression tasks: Use a single neuron with linear activation.
     - Classification tasks: Use softmax for multi-class or sigmoid for binary classification.
3. **Initialize Weights and Biases**
   - Initialize weights using Xavier or He initialization.
   - Biases are typically initialized to small values (e.g., zero).
4. **Forward Propagation**

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

   Repeat until the output layer is reached.
5. **Loss Calculation**

Choose an appropriate loss function:

- Mean Squared Error (MSE) for regression.
- Cross-Entropy Loss for classification.

6. Backpropagation

- Compute error and gradients with respect to weights and biases:

$$\frac{\partial L}{\partial W^{(l)}} = a^{(l-1)}\delta^{(l)}, \quad \frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}$$

- Update weights:

$$W^{(l)} = W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}}, \quad b^{(l)} = b^{(l)} - \eta \frac{\partial L}{\partial b^{(l)}}$$

7. Training

- Train the model using optimization algorithms like SGD, Adam, or RMSProp.
- Use validation data to monitor performance and avoid overfitting.

8. Evaluate Performance

- Use metrics such as accuracy, precision, recall, F1-score (classification) or RMSE (regression).
- Visualize performance using plots or confusion matrices.

9. Prediction

Use the feedforward phase to make predictions on new data.

3.2 Combined Actuarial Neural Network (CANNs)

CANNs combine classical actuarial models (e.g., GLMs) with neural networks to enhance predictive accuracy while maintaining interpretability.

1. Data Preparation

- Collect structured insurance datasets including policyholder features and claim counts.
- Preprocess Data:
  - Normalize numerical features.
  - Encode categorical variables.
  - Split the dataset into training, validation, and test sets.

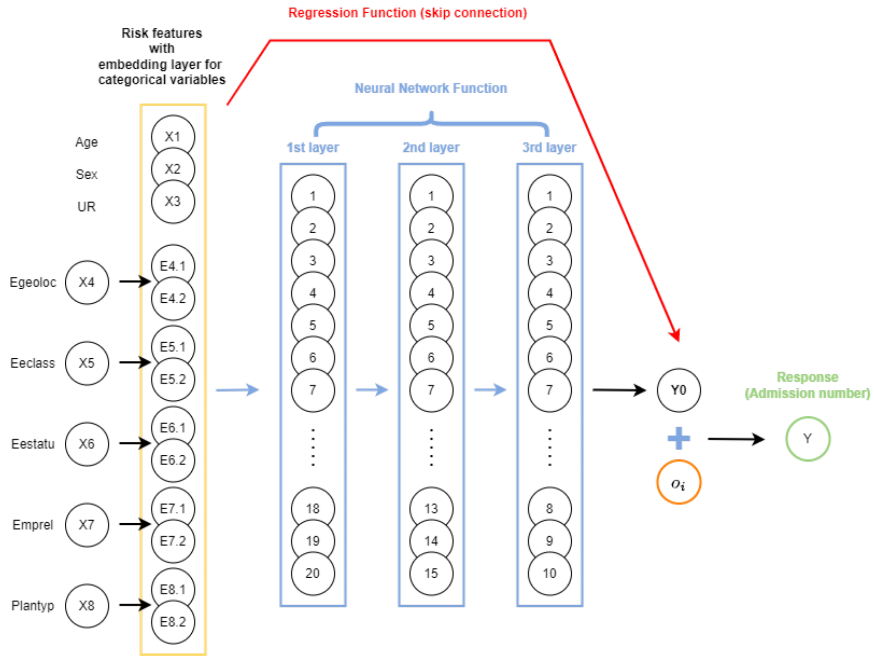

Figure 5: Steps to Build a Combined Actuarial Neural Network (CANN)

2. Define the CANN Architecture

Classical Regression Model (GLM):

$$\mu_{\text{GLM}}(x;\beta) = \exp(\beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n)$$

Neural Network Component:

$$\mu_{\text{NN}}(x;\theta) = f(Wx + b)$$

Blending the Two Components:

$$\mu_{\text{CANN}}(x;\beta,\theta) = \mu_{\text{GLM}}(x;\beta) + \mu_{\text{NN}}(x;\theta)$$

Alternatively:

$$\mu_{\text{CANN}}(x;\beta,\theta) = \mu_{\text{GLM}}(x;\beta) \times \mu_{\text{NN}}(x;\theta)$$

3. Initialize Parameters

- Estimate GLM parameters β using Maximum Likelihood Estimation.
- Initialize neural network weights θ using Xavier or He initialization.

4. Training Process

Step 1: Loss Function

$$L = -\sum_{i=1}^{n} (y_i \log(\mu_i) - \mu_i)$$

Step 2: Optimization

$$W^{(l)} = W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}}$$

Step 3: Regularization

- Use L2 regularization or dropout to prevent overfitting.

5. Hyperparameter Tuning

Tune:

- Number of neurons
- Learning rate
- Batch size
- Number of epochs
- Optimizer type (Adam, SGD, etc.)

6. Validation and Testing

Use RMSE, Poisson deviance loss, or other metrics to compare with standalone models.

7. Prediction

$$\hat{\mu}_{\text{CANN}}(x) = \mu_{\text{GLM}}(x; \hat{\beta}) + \mu_{\text{NN}}(x; \hat{\theta})$$

Advantages of CANNs

- **Interpretability:** GLM provides clear actuarial insights.
- **Flexibility:** Neural network captures complex, nonlinear relationships.
- **Enhanced Accuracy:** The hybrid model improves predictive performance.

4 Parameters and Hyperparameters

Understanding the distinction between **parameters** and **hyperparameters** is crucial for designing, training, and optimizing neural networks. While parameters are learned during training, hyperparameters are defined prior to training and guide the learning process.

4.1 Feedforward Neural Networks

Feedforward Neural Networks (FNNs) rely on both parameters and hyperparameters to effectively learn from data and generalize on unseen samples.

1. Parameters

These are internal variables adjusted during training through backpropagation.

- **Weights:** Connect neurons between layers. They represent the strength of input-output relationships and are updated using gradient-based optimizers.
- **Biases:** Added to the weighted input before applying activation functions. They allow the model to better fit the data by shifting activations.

Proper initialization (e.g., Xavier or He) ensures faster convergence and avoids issues such as vanishing or exploding gradients.

2. Hyperparameters

These define the architecture and behavior of the training process. They are manually set prior to training.

- **Learning Rate:** Controls the step size during gradient descent. A smaller value ensures stability, while a larger value accelerates convergence.
- **Batch Size:** Number of samples processed before updating weights. Smaller batches lead to noisier gradients; larger batches improve efficiency.
- **Number of Hidden Layers:** Adds depth to the network. Deeper models capture more complex features but may overfit.
- **Neurons per Layer:** Determines the learning capacity. Too few neurons may underfit; too many may overfit.
- **Activation Functions:** Introduce non-linearity (e.g., ReLU, sigmoid, tanh). ReLU is common in hidden layers.
- **Weight Initialization:** Xavier or He initialization helps balance gradients.
- **Regularization:** Dropout and L1/L2 penalties reduce overfitting.
- **Optimization Algorithm:** Methods like SGD, Adam, and RMSProp govern how gradients update weights.
- **Epochs:** Full passes through the dataset. Requires balancing to prevent underfitting or overfitting.

Hyperparameter Tuning Strategies:

- Grid Search or Random Search
- Bayesian Optimization or Hyperband
- Empirical Testing with a validation set or k-fold cross-validation

Table 1: Parameters vs. Hyperparameters

Type	**Examples**	**Purpose**
Parameters	Weights, Biases	Learned by the model during training
Hyperparameters	Learning Rate, Batch Size, Epochs	Set externally to control training behavior

4.2 Combined Actuarial Neural Networks (CANNs)

1. Parameters

These are internal variables adjusted during training through backpropagation.

- **Weights and Biases:** Connect neurons between layers. They represent the strength of input-output relationships and are updated using gradient-based optimizers.
- **Coefficients in GLM:** Estimated using Maximum Likelihood Estimation (MLE). These coefficients represent the baseline estimation for the actuarial model.

2. Hyperparameters

These define the architecture and behavior of the training process. They are manually set prior to training.

- **Learning Rate:** Controls the step size during gradient descent.
- **Batch Size:** Number of samples processed before updating weights.
- **Number of Hidden Layers and Neurons:** Determines the model's capacity to learn complex patterns.
- **Activation Functions:** Introduce non-linearity in hidden layers (e.g., ReLU, sigmoid).
- **Regularization Techniques:** Dropout, L1/L2 penalties reduce overfitting.
- **Optimization Algorithm:** Methods like Adam, SGD, or RMSProp.
- **Epochs:** Full passes through the dataset during training.
- **Weight Initialization:** Techniques like Xavier or He initialization.
- **Embedding Dimension:** The size of the embedding layer for categorical features.
- **Dropout Rates:** Controls the proportion of neurons deactivated during training.

Hyperparameter Tuning Strategies:

- Grid Search or Random Search
- Response Surface Methodology (RSM)
- Genetic Algorithm
- Empirical Testing with a validation set

References

Olsgärde, N. (2021). Deep bayesian neural networks for prediction of insurance premiums. *Probabilistic Deep Learning*.